



Vue.js 입문 강의

프론트엔드에서 널리 쓰이는 프레임워크, Vue.js 시작하기

Vue.js 강의 커리큘럼

전체 강의 흐름 로드맵

1. Vue.js 시작

개발환경 설정 및 기본기

2. Vue.js 뼈대

컴포넌트, props, event

3. 상태관리 및 반응형

computed, watch, pinia

4. 중간 예제 실습

TODO 어플리케이션 구현

5. Vue Router

라우팅 및 페이지 관리

6. 실전 활용

API 연동과 백엔드 연계

7. 차트 시각화 및 반응형 UI

데이터 시각화 기법

8. Firebase 연동

auth, firestore

9. 실전 프로젝트

완성된 기술로 실제 애플리케이션 구현

개발 환경 구축



Node.js 설치

Vue 프로젝트를 실행하는 엔진, LTS 버전 설치 권장

```
node -v
```

```
npm -v
```



VSCode 설치

코드 에디터로 VSCode 권장, Volar 확장 프로그램 필수 설치



Vue CLI 설치

Vue 프로젝트 생성 도구 설치

```
npm install -g @vue/cli
```

```
vue --version
```

개발 환경 구축



프로젝트 생성 실습

```
vue create my-app
```

```
cd my-app
```

```
npm run serve
```

접속: localhost:8080

참고: 최신 권장 초기화 도구는 *create-vue*(Vite 기반)

대안: *npm create vue@latest*

개발 환경 구축



Welcome to Your Vue.js App

For a guide and recipes on how to configure / customize this project,
check out the [vue-cli documentation](#).

Installed CLI Plugins

[babel](#) [eslint](#)

Essential Links

[Core Docs](#) [Forum](#) [Community Chat](#) [Twitter](#) [News](#)

Ecosystem

[vue-router](#) [vuex](#) [vue-devtools](#) [vue-loader](#) [awesome-vue](#)

첫 소스 살펴보기



Welcome to Your Vue.js App

For a guide and recipes on how to configure / customize this project, check out the [vue-cli documentation](#).

Installed CLI Plugins

[babel](#) [eslint](#)

Essential Links

[Core Docs](#) [Forum](#) [Community Chat](#) [Twitter](#) [News](#)

Ecosystem

[vue-router](#) [vuex](#) [vue-devtools](#) [vue-loader](#) [awesome-vue](#)

```
<template>
  
  <HelloWorld msg="Welcome to Your Vue.js App"/>
</template>

<script>
import HelloWorld from './components/HelloWorld.vue'

export default {
  name: 'App',
  components: {
    HelloWorld
  }
}
</script>
```

첫 소스 살펴보기



`<script>` **VS** `<script setup>`

Vue 3 Composition API 문법 비교

템플릿 문법: 데이터 바인딩

중괄호 {{ }} 표현식을 활용한 반응형 데이터 출력

기본 데이터 바인딩

중괄호 두 개 {{ }} 안에 변수를 넣으면 화면에 자동으로 출력됩니다.

```
<script setup>
const message = 'Welcome to Vue!'
</script>
<template>
<h2>{{ message }}</h2>
</template>
```

Welcome to Vue!

템플릿 문법: 데이터 바인딩



반응형 동작

데이터가 변경되면 관련된 모든 화면 요소가 자동으로 업데이트됩니다.



표현식 사용 가능

```
<template>
<p>{{ message.split('').reverse().join('') }}</p>
<p>{{ 1 + 2 * 3 }}</p>
</template>
```

!euv ot emoclew

7

입력 바인딩: v-model로 양방향 데이터 연결

사용자 입력과 데이터가 자동으로 연결되는 양방향 바인딩



기본 v-model 사용법

사용자의 입력 값이 변수에 자동으로 매핑됨 (양방향 바인딩)



v-model 수식어(Modifiers)

.number - 사용자 입력을 자동으로 숫자 타입으로 변환

!euv ot emoclew

7

welcome to vue!

이름

나이

홍길동 (5)

```
<div>
  <label>이름</label>
  <input v-model="name" placeholder="이름을 입력" />
  <p></p>
  <label>나이</label>
  <input v-model.number="age" type="number" placeholder="숫자만" />
  <p>{{ name }} ({{ age }}) </p>
</div>
```

```
<script setup>
// import HelloWorld from './components/HelloWorld.vue'
import { ref } from 'vue'

const message = 'welcome to vue!'

const name = ref('')
const age = ref(0)
</script>
```

ref vs reactive: 개념과 차이

Vue의 반응형 시스템의 두 가지 핵심 API 비교

ref

```
import { ref } from 'vue'
// 원시값(숫자, 문자열, 불리언)
const count = ref(0)
count.value++
// 객체도 가능
const person = ref({ name: 'Kim' })
person.value.name = 'Lee'
// 객체 자체 교체 가능
person.value = { name: 'Park' }
```

reactive

```
import { reactive } from 'vue'
// 객체 또는 배열만 가능
const state = reactive({
  user: { name: 'Kim' },
  todos: ['Vue 학습']
})
// 속성 변경
state.user.name = 'Lee'
state.todos.push('Vuex 학습')
// 객체 자체 교체 불가능
// state = {} // ❌ 에러
```

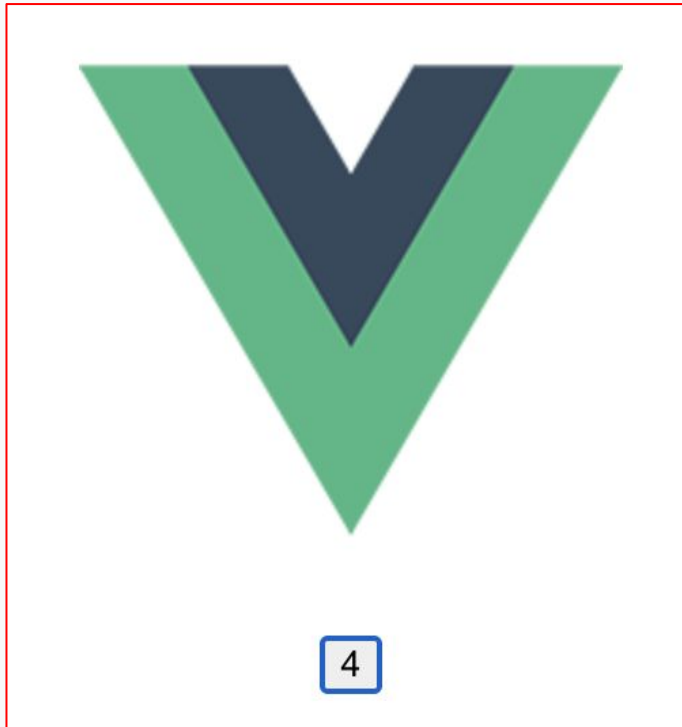
핵심 차이점

구분	ref	reactive
데이터 타입	모든 타입 가능	객체와 배열만 가능
접근 방식	.value로 접근 (템플릿에서는 자동 언래핑)	직접 속성 접근
교체 가능성	참조 자체 교체 가능	참조 자체 교체 불가 (내부 속성만 변경)
구조 분해	toRefs 없이 분해 시 반응성 손실	toRefs 사용 필요

템플릿 문법: 데이터 바인딩 연습

Composition API에서 반응형 상태를 선언하는 권장 방법은 `ref()` 함수를 사용하는 것입니다:

컴포넌트가 처음 렌더링될 때, **Vue**는 렌더링 과정에서 사용된 모든 **ref**를 추적합니다. 나중에 **ref**가 변경되면, 이를 추적하는 컴포넌트에 대해 재렌더링을 **트리거** 합니다.



```
<template>
  
  <p></p>
  <button @click="increase">
    | | | {{ count }}
  </button>
</template>

<script setup>
import { ref } from 'vue'

const count = ref(0)

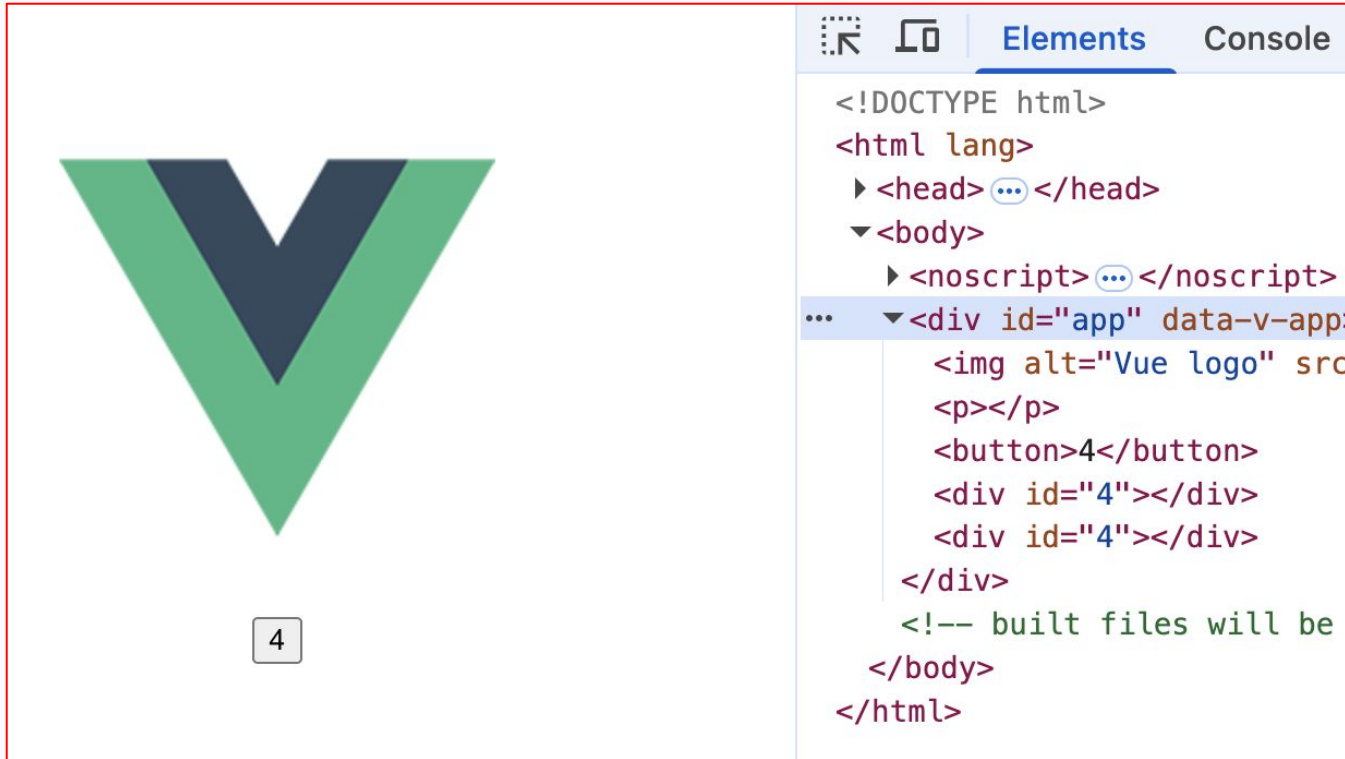
const increase = () => {
  count.value += 1
}

// function increase(){
//   count.value += 1
// }
</script>
```

Vue 함수 선언 방식 비교

Composition API 기준으로 본 화살표 함수 vs 일반 함수

템플릿 문법: 데이터 바인딩 연습



주의사항

중괄호 표현식은 텍스트 노드에서만 사용 가능합니다.

속성 바인딩에는 v-bind 또는 단축 문법인 :를 사용해야 합니다.

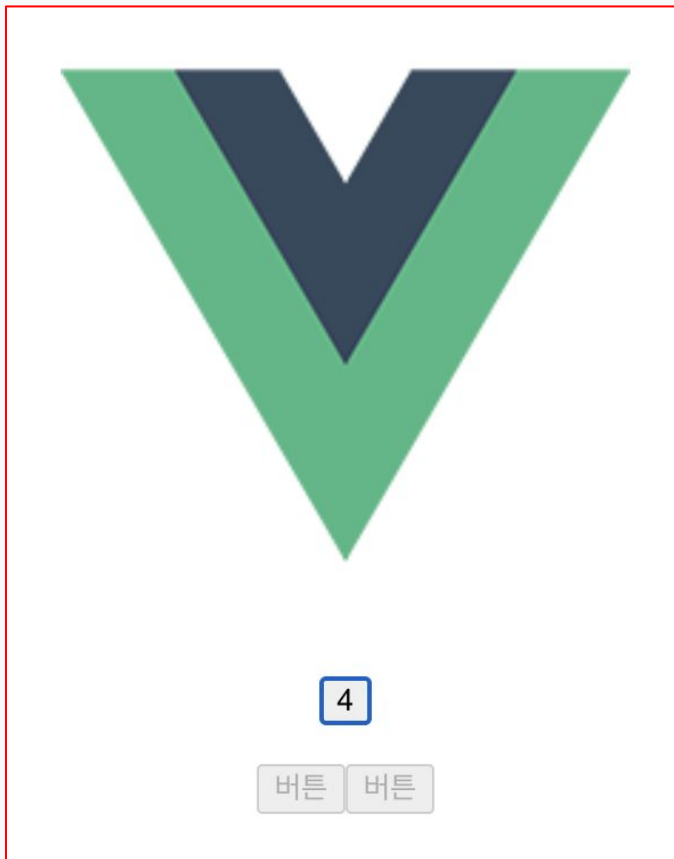
템플릿 문법: 데이터 바인딩 연습



```
<!-- ref로 상태 제어 -->  
<button :disabled="isButtonDisabled">  
  버튼  
</button>  
  
<!-- 조건식 직접 바인딩 -->  
<button :disabled="count > 3">  
  버튼  
</button>
```

```
<script setup>  
import { ref } from 'vue'  
const isButtonDisabled = ref(true)
```

동적 클래스 바인딩 연습(v-bind:class Binding)



```
<img  
  alt="Vue logo"  
  :class="{ logo: count > 3 }"  
  src="./assets/logo.png"  
  width="125"  
  height="125"  
>
```

```
<style>  
.logo {  
  display: block;  
  margin: 0 auto 2rem;  
  width: 250px;  
  height: 250px;  
}  
</style>
```


템플릿 문법: v-if 조건 렌더링

조건에 따라 요소를 표시하거나 숨기는 방법

기본 문법

v-if 디렉티브는 조건이 true일 때만 요소를 렌더링합니다.

```
<template>  
<p v-if="true">보입니다</p>  
<p v-if="false">안 보입니다</p>  
</template>
```

결과:

보입니다

(두 번째 요소는 렌더링되지 않음)

템플릿 문법: v-if 조건 렌더링

조건에 따라 요소를 표시하거나 숨기는 방법



조건식 사용

변수나 복잡한 표현식도 사용할 수 있습니다:

```
<script setup>
const isLoggedIn = true;
const userRole = 'admin';
</script>
<template>

<div v-if="isLoggedIn">환영합니다!</div>
<div v-if="userRole === 'admin'">관리자 메뉴</div>
</template>
```

템플릿 문법: v-if 조건 렌더링

조건에 따라 요소를 표시하거나 숨기는 방법



v-else / v-else-if 함께 사용하기

```
<template>
<div v-if="type === 'A'">A 타입입니다</div>
<div v-else-if="type === 'B'">B 타입입니다</div>
<div v-else>A도 B도 아닙니다</div>
</template>
```

주의: v-else와 v-else-if는 반드시 v-if 요소 바로 다음에 와야 합니다.

v-if는 조건이 바뀔 때 요소를 완전히 제거하고 다시 생성합니다.

자주 토글되는 요소에는 v-show 디렉티브 사용을 고려하세요. (CSS display 속성으로 숨김)

템플릿 문법: v-if 조건 렌더링

실습

8

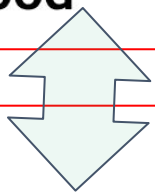
버튼

버튼

Vue는 멋지죠!

Vue는 정말 멋지죠!

Vue is Good



9

버튼

버튼

아닌가요? 🥲

Vue is Bad

```
<!-- v-show -->
<h1 v-show="isButtonDisabled">
  Vue는 멋지죠!
</h1>
```

```
<!-- v-if / v-else -->
<h1 v-if="isButtonDisabled">
  Vue는 정말 멋지죠!
</h1>
<h1 v-else>
  아닌가요? 🥲
</h1>
```

```
<!-- 삼항 연산자 -->
<h2>
  {{ isButtonDisabled ? 'Vue is Good' : 'Vue is Bad' }}
</h2>
```

```
const isButtonDisabled = ref(true)
```

```
const count = ref(0)
```

```
const increase = () => {
  count.value += 1
  isButtonDisabled.value = !isButtonDisabled.value
}
```

템플릿 문법: v-for 반복 렌더링

리스트 출력을 위한 v-for 디렉티브와 :key 속성의 활용



리스트 반복 출력

배열 데이터를 화면에 목록으로 렌더링할 때 사용하는 v-for 디렉티브

```
<script setup>
const items = ['Apple', 'Banana', 'Cherry']
</script>
<template>
<ul>
<li v-for="(fruit, i) in items" :key="i">
  {{ i }} - {{ fruit }}
</li>
</ul>
</template>
```



:key 속성의 중요성

각 항목을 고유하게 식별하기 위한 key 속성 필수 사용

- DOM 요소 재사용 최적화
- 항목 순서 변경 시 정확한 렌더링
- 성능 향상 및 예측 가능한 동작

템플릿 문법: v-for 반복 렌더링

리스트 출력을 위한 v-for 디렉티브와 :key 속성의 활용



v-for 활용 예시

```
// 객체 반복
<div v-for="(value, key) in user" :key="key">
  {{ key }}: {{ value }}
</div>
// 숫자 범위 반복
<span v-for="n in 5" :key="n">{{ n }} </span>
```

출력 결과:

0 - Apple
1 - Banana
2 - Cherry

v-for와 v-if를 같은 요소에 함께 사용하지 않는 것을 권장합니다.

→v-if가 v-for보다 우선순위가 높아 예상치 못한 동작이 발생할 수 있습니다.

템플릿 문법: v-for 반복 렌더링

실습

배열 반복

- 0 - Apple
- 1 - Banana
- 2 - Cherry

객체 반복

name : Alice
age : 25
city : Seoul

숫자 반복

12345

```
<!-- 🍏 배열 반복 -->
<h2>🍏 배열 반복</h2>
<ul>
  <li v-for="(fruit, i) in items" :key="i">
    {{ i }} - {{ fruit }}
  </li>
</ul>

<!-- 👤 객체 반복 -->
<h2>👤 객체 반복</h2>
<div v-for="(value, key) in user" :key="key">
  {{ key }} : {{ value }}
</div>

<!-- 📄 숫자 범위 반복 -->
<h2>📄 숫자 반복</h2>
<span v-for="n in 5" :key="n">
  {{ n }}
</span>
```

```
const items = ['Apple', 'Banana', 'Cherry']

const user = {
  name: 'Alice',
  age: 25,
  city: 'Seoul'
}
```

핵심 개념 정리

Vue.js의 주요 특징과 작동 원리



반응형 시스템 (ref)

데이터가 변경되면 화면이 자동으로 갱신되는 핵심 메커니즘

변수 선언만으로 양방향 바인딩이 가능하며, 상태 관리가 직관적



컴포넌트 기반 개발

재사용 가능한 독립적 모듈로 UI 구성

SFC(Single File Component)로 HTML, JS, CSS를 한 파일에서 관리



템플릿 문법

직관적인 디렉티브 시스템으로 DOM 조작

텍스트 보간법 {{ }}, 속성 바인딩 :, 이벤트 처리 @, 조건부 v-if, 반복문 v-for

마무리 & 다음 강의 예고

오늘 배운 내용 정리 및 다음 강의 준비



오늘 정리

환경 구축 - Node.js, VSCode, Vue CLI 설치

첫 컴포넌트 - Single File Component 생성 및 렌더링

템플릿 문법 - {{ }}, v-if, v-for 기본 문법 학습

프로젝트 실행 - Vue 앱 생성 및 localhost에서 실행



다음 강의 예고

컴포넌트 구조 설계